

Binary Token Transmission with Client-Side Detokenization for Language Model Inference

Harjinder Singh* | Independent Researcher |  code

May 11, 2026

Abstract. Streaming inference from language models typically transmits decoded text with JSON serialization overhead. This paper evaluates an alternative format: transmitting token identifiers as fixed-size binary values, with text decoding performed client-side. Using Phi-2 (2.7B parameters) across 6 runs with 30 prompts each, we compare JSON-encoded text against 4-byte binary transmission. Binary transmission reduces payload by 92.2% (14.3 KB vs 182.2 KB for 3,650 tokens), a reduction fixed by protocol design (4 bytes per token) relative to our measured JSON baseline (51 bytes per token average). Time-to-first-token distributions overlap between formats, with no practically meaningful separation in our local execution environment. This study specifies a simple binary token transmission protocol and quantifies its payload trade-off relative to a JSON baseline under a local measurement setup. Whether these payload reductions translate into end-to-end latency or cost benefits under networked deployment remains an open question for future work.

1 Introduction

Language model inference commonly streams output tokens as they are generated, reducing perceived latency. The typical pattern involves server-side detokenization followed by JSON serialization for each token. This format prioritizes web compatibility but introduces per-token overhead.

This work evaluates an alternative: transmitting raw token identifiers as fixed-width binary values, with the client performing detokenization using a locally stored vocabulary dictionary.

Our evaluation uses Phi-2 [Microsoft Research, 2023], a 2.7 billion parameter model. We compare two transmission formats: a JSON baseline matching common API patterns, and a binary format transmitting 4-byte token identifiers. The binary approach requires distributing a vocabulary dictionary to clients; for Phi-2, our implementation produces a dictionary of 0.8 MB.

We present measurements across 6 benchmark runs totaling 360 prompt executions. Binary transmission reduces payload by 92.2%, a ratio determined by the fixed 4-byte protocol design versus our measured JSON payload sizes averaging 51 bytes per token. Time-to-first-token distributions overlap in our local setup.

This paper documents the protocol, quantifies the trade-off, and scopes what remains unmeasured.

2 Background

2.1 Language Model Inference

Autoregressive language models generate output one token per forward pass. Each token is an integer identifier from a fixed vocabulary, converted to text through detokenization before user presentation.

Models such as Phi-2 [Microsoft Research, 2023], Llama [Touvron et al., 2023], and Qwen [Bai et al., 2023] execute in quantized form on consumer hardware via inference engines like llama.cpp [Gerganov

*Corresponding author

and contributors, 2023].

2.2 Tokenization

Subword tokenization algorithms—Byte-Pair Encoding [Sennrich et al., 2016], WordPiece [Wu et al., 2016], SentencePiece [Kudo and Richardson, 2018]—produce vocabularies mapping integer identifiers to text fragments. Phi-2 uses a vocabulary of 51,200 tokens.

2.3 Streaming Protocols

Current streaming implementations commonly use text-oriented transport, such as Server-Sent Events or WebSocket messages carrying JSON payloads. In our baseline, a single-token payload containing model identifier, decoded text, and completion status consumes 45–70 bytes depending on content.

Schema-based formats such as Protocol Buffers [Google, 2023] or MessagePack [Furuhashi, 2023] reduce overhead versus JSON but retain field tags and length delimiters. For transmitting a single known-width value, raw fixed-width binary avoids this schema overhead entirely. We adopt 32-bit unsigned integers as an engineering choice providing constant-width representation, straightforward implementation, and headroom beyond current vocabulary sizes—not as an information-theoretic optimum.

2.4 Related Work

Efficient inference has been studied through speculative decoding [Leviathan et al., 2023], memory-efficient attention [Kwon et al., 2023], and edge-cloud distribution [Laskaridis et al., 2020]. Our work differs in focus: we measure transmission format overhead rather than inference optimization.

3 Method

3.1 Transmission Formats

We compare two formats for streaming token output.

JSON baseline. Each token is decoded to text and wrapped in a JSON object:

```
{"model": "llm", "response": "<text>", "done": false}
```

Payload size varies with token content. Non-ASCII characters may incur Unicode escaping (\uXXXX), increasing size further.

Binary format. Each token identifier is transmitted as a 4-byte value. The client performs detokenization using a pre-loaded vocabulary dictionary.

3.2 Binary Protocol Specification

- **Data type:** 32-bit unsigned integer (uint32)
- **Byte order:** Little-endian
- **Size:** 4 bytes per token, including special tokens (BOS, EOS, PAD)

The 32-bit width is an engineering choice: it accommodates vocabularies up to 2^{32} tokens with low implementation complexity. Current models use vocabularies under 2^{17} ; a 16-bit format would suffice for these but would constrain future vocabulary growth.

3.3 Implementation Context

Our experiments execute locally: the model generates tokens, and we record the byte counts that *would* be transmitted under each format. No actual network transport occurs.

In a networked deployment, the same binary representation could be carried either in WebSocket binary messages or over a raw byte stream. In the latter case, clients would buffer incoming bytes until complete 4-byte boundaries are available. We did not implement or measure networked transport in this study.

For dictionary synchronization, we propose a handshake in which clients send a dictionary hash with requests; servers respond with updated dictionaries in case of mismatch. This remains unimplemented.

3.4 Experimental Setup

We used Phi-2 [Microsoft Research, 2023] in Q4_K_M quantized format via llama.cpp [Gerganov and contributors, 2023] on an NVIDIA T4 GPU (Google Colab). The vocabulary contains 51,200 tokens.

Our corpus comprises 30 prompts:

- Code generation: 14 prompts
- Technical reasoning: 14 prompts
- Creative writing: 2 prompts

Each prompt was limited to 128 output tokens. We executed 6 complete runs testing both formats on all prompts, yielding 360 total executions with warmup and cooldown phases.

3.5 Dictionary Format

The client dictionary uses a binary layout:

- Header: 4-byte magic, 4-byte entry count
- Offset table: $51,200 \times 4$ -byte offsets
- String data: concatenated UTF-8 token strings

The search is $O(1)$. For Phi-2, this produces a 0.8 MB file. Clients can load it via memory-mapped I/O to avoid parsing overhead.

4 Results

4.1 Payload Size

Table 1 summarizes the payload measurements.

Table 1: Payload comparison for 3,650 tokens. Binary size is fixed by protocol; JSON size is measured from our implementation.

Metric	JSON Baseline	Binary Format
Total payload	182.2 KB	14.3 KB
Per-token average	51.1 bytes	4.0 bytes
Reduction	92.2%	

Binary payload values are deterministic under the specified protocol, whereas JSON payload values reflect the measured behavior of our implemented baseline. The 92.2% reduction is the ratio of these quantities.

Figure 1 presents the comparison.

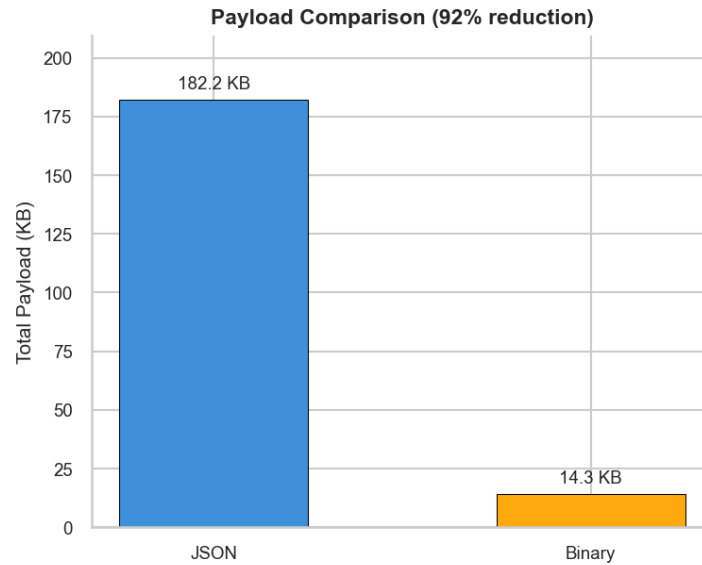


Figure 1: Payload comparison for 3,650 tokens. Binary transmission uses a fixed 4-byte representation (14.3 KB total), whereas the JSON baseline averages 51 bytes per token (182.2 KB total).

4.2 Time to First Token

Table 2 presents TTFT aggregated across 6 runs.

Table 2: Time to first token (ms) across 6 runs, 180 measurements per format.

Statistic	JSON Baseline	Binary Format
Mean	20.58	20.31
Std Dev	1.32	1.21
95% CI	[19.19, 21.97]	[19.05, 21.58]

Confidence intervals overlap substantially. In our local execution environment, we did not observe a practically meaningful TTFT separation between formats. This is expected because both formats share the same inference path; they differ only in post-generation payload construction and byte accounting, both of which occur in local memory.

4.3 Category Breakdown

Table 3 shows consistency across prompt categories.

Table 3: Payload by prompt category.

Category	Prompts	Tokens	JSON (KB)	Reduction
Code	14	1,694	84.2	92.1%
Reasoning	14	1,792	91.8	92.3%
Creative	2	164	6.2	92.1%

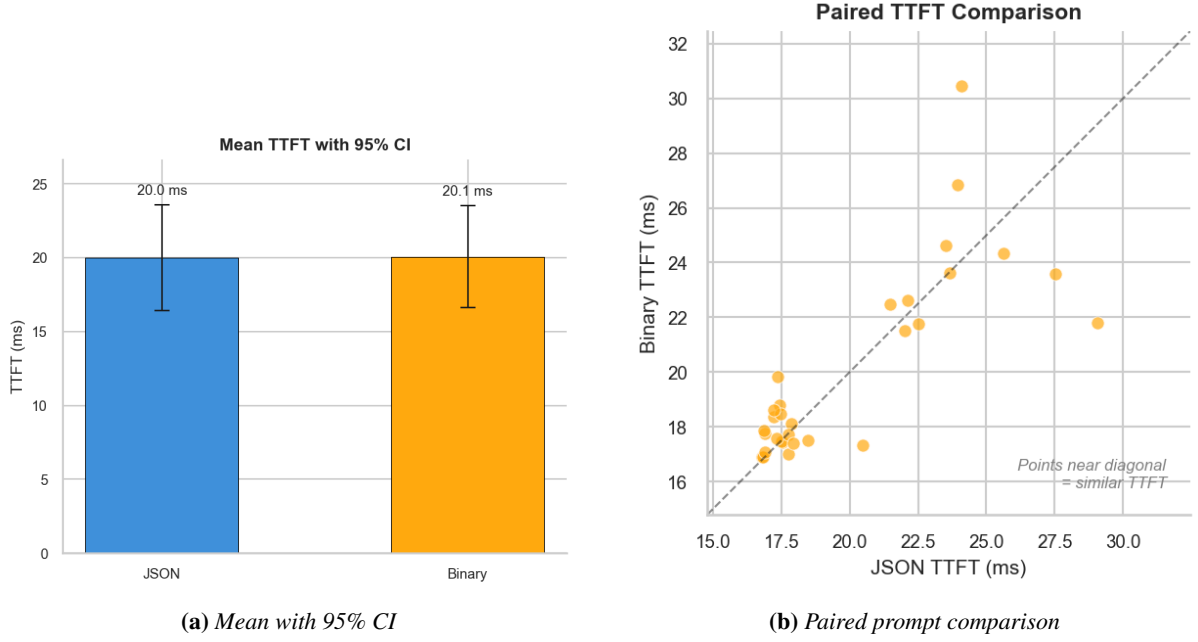


Figure 2: TTFT analysis. (a) Overlapping intervals. (b) Each point is one prompt; diagonal line indicates parity. No practically meaningful separation observed in this local setup.

5 Discussion

5.1 Break-Even Analysis

Binary transmission requires a one-time dictionary download. Given dictionary size $D = 800$ KB and per-token savings $S = 47.1$ bytes:

$$N_{\text{break-even}} = \frac{D}{S} = \frac{819,200}{47.1} \approx 17,400 \text{ tokens} \quad (1)$$

At 128 tokens per response, this is approximately 136 sessions. Applications with higher per-session token counts reach break-even faster.

5.2 Internationalization

Our benchmark used predominantly ASCII text. For CJK or emoji-heavy content, the JSON baseline may incur larger payloads because escaped or multi-byte text fragments occupy more bytes, whereas the binary representation remains fixed at 4 bytes per token.

5.3 Scope and Limitations

The following remain unmeasured in this study:

- **Network transmission.** All measurements are local byte counts. Latency, throughput, and congestion effects under actual network conditions are not characterized.
- **Compression.** We did not measure HTTP compression (gzip, brotli) on either format.
- **Detokenization fidelity.** We did not verify that client-side dictionary output matches server-side tokenizer output for all sequences.
- **Other models.** We tested Phi-2 only. Dictionary sizes vary with vocabulary size.

6 Conclusion

Under our implemented baseline, binary token transmission reduced payload by 92.2% compared to JSON, a ratio of the fixed 4-byte protocol size to the measured 51-byte JSON average. In our local setup, we did not observe a stable TTFT advantage for either format. The binary approach requires client-side dictionary storage (0.8 MB for Phi-2) with a break-even point of approximately 17,400 tokens.

Whether this payload reduction yields end-to-end latency or cost benefits under actual network conditions remains an open question for future measurement.

Reproducibility

Benchmark code, including a Colab notebook and figure-generation script, is provided. The binary protocol and dictionary format are fully specified, enabling independent implementation.

References

- Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. Qwen technical report. *arXiv preprint arXiv:2309.16609*, 2023.
- Sadayuki Furuhashi. *MessagePack*, 2023. URL <https://msgpack.org/>. Binary serialization format.
- Georgi Gerganov and contributors. llama.cpp. <https://github.com/ggerganov/llama.cpp>, 2023. Open-source LLM inference in C/C++.
- Google. *Protocol Buffers*, 2023. URL <https://protobuf.dev/>. Language-neutral data serialization.
- Taku Kudo and John Richardson. SentencePiece: A simple and language independent subword tokenizer and detokenizer for neural text processing. In *Proceedings of EMNLP: System Demonstrations*, pages 66–71, 2018.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, 2023.
- Stefanos Laskaridis, Stylianos I. Venieris, Mario Almeida, Ilias Leontiadis, and Nicholas D. Lane. SPINN: Synergistic progressive inference of neural networks over device and cloud. In *Proceedings of MobiCom*, 2020.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286, 2023.
- Microsoft Research. Phi-2: The surprising power of small language models. Technical report, Microsoft, December 2023. Technical Report.
- Rico Sennrich, Barry Haddow, and Alexandra Birch. Neural machine translation of rare words with subword units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*, pages 1715–1725, 2016.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V. Le, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.